

ASSIGNMENT**1. What are the differences between Native, Hybrid, and Cross-platform mobile application development, and when is each approach commonly used?**

- Native apps run fastest since they're built separately per OS, so they fit games and hardware-heavy apps, hybrid apps are websites in an app shell, cheap and quick for tight budgets, and cross-platform writes once but compiles close to native speed, which is why most ordinary apps use it now. Flutter is Google's cross-platform toolkit, built on Dart with everything as a widget, hot reload for instant changes, and a big package library, compiling to native code instead of running through a browser. None of that works well without SDLC though, the plan-design-build-test-launch-maintain cycle, since skipping a step usually shows up later as a bug or a wasted feature. Flutter actually speeds up that whole cycle instead of replacing it, design maps to widgets, testing tools are built in, one codebase covers deployment and maintenance for both platforms. So knowing all this before starting a project is what separates someone who just codes from someone who builds something that actually lasts.

2. What is the Flutter Ecosystem, and how does it help developers create modern mobile applications?

- Flutter is Google's toolkit for building apps across mobile, web, and desktop from one codebase, built around a few core pieces working together. Dart is the language, fast and easy to pick up, the widget library gives ready-made visual blocks for any screen size, the Flutter engine is the C++ core actually drawing graphics on the device, and pub.dev is where the community shares plugins so nobody starts from zero. Together these let a developer write once and ship everywhere, see changes appear instantly through hot reload, get near-native speed since Dart compiles straight to machine code, and still have full pixel-level control for a custom look instead of settling for generic templates. That mix of speed, reach, and design freedom is basically why Flutter feels less like a shortcut and more like a real toolkit for modern app building.

3. What is the Software Development Life Cycle (SDLC), and why is it important in building successful applications?

- SDLC is the structured process teams follow to take software from idea to finished product, planning, analysis, design, coding, testing, deployment, and maintenance, each phase building on the one before it. Planning sets the scope and goals, analysis digs into what's actually needed, design maps out the architecture and interface, development is where the code gets written, testing

catches what's broken before users ever see it, deployment puts it live, and maintenance keeps it working long after launch. What makes this matter isn't just structure for its own sake, it's that a bug caught in design costs almost nothing compared to one caught after release, testing keeps the product actually reliable instead of just functional, and clear phases give everyone, developers, testers, and stakeholders, visibility into progress so the whole team stays pointed at the same goal instead of working in different directions.

4. How do Flutter development practices and the SDLC work together in creating real-world mobile applications?

- Flutter doesn't replace the SDLC, it just makes each phase move faster and smoother. During planning and design, developers can already imagine the UI as widgets, since Flutter's structure practically mirrors how design systems are built. During development, hot reload turns what used to be a slow build-test-repeat cycle into something almost instant, so teams can experiment without losing momentum. Testing also benefits, since Flutter has built-in tools for unit, widget, and integration testing, which fits naturally into the testing phase of SDLC instead of needing separate tools. And when it's time to deploy, one codebase can go to both Android and iOS stores, cutting the deployment phase almost in half. Even maintenance is easier, since bug fixes and updates only need to happen once, not twice. So really, Flutter is the tool, and SDLC is the discipline guiding how that tool gets used properly instead of carelessly.

5. Why should developers understand these concepts before starting an application project?

- Understanding these ideas before writing a single line of code is what saves a project from falling apart later, since going back to fix a bad architecture decision or a security gap after launch costs way more than planning for it upfront. Knowing your architecture, how the app's pieces talk to each other, whether that's microservices or a simpler monolithic setup, shapes how easily the app grows. Knowing your data flow keeps information moving and saving safely instead of breaking under pressure. Thinking about security early, logins, encryption, safe APIs, stops problems before they become breaches, not after. And keeping resource limits, server costs, speed, storage, in mind from day one keeps the app usable as more people start using it, not just functional for a demo. All of it comes down to one thing really: a developer who plans first writes cleaner code, works better with a team, and builds something that can actually scale instead of something that just barely works.